Git command summary

Robin Engler – 2026-11-03

Configuring Git **2**
- User name and email . . . 2
- Change default editor . . . 2
- Creating Git command aliases . . . 2

Initializing a new repo and making commits **3**
- Create a new Git repository . . . 3
- Staging: add and remove content from the index . . . 3
- Making new commits . . . 3
- Amend the latest commit of the current branch . . . 3
- Display file status, show content differences and history . . . 3
- Ignoring files . . . 4

Branches **4**
- Create and switch branches . . . 4
- Delete branches . . . 4
- Merge, rebase and cherry-pick . . . 5

Retrieve content from earlier commits **5**

Working with remotes **5**
- Fetch/pull new content from a remote . . . 5
- Push new content to a remote . . . 6
- Add or update a remote . . . 6

Relative references to commits **6**

Detached HEAD state **7**

Editing the Git history **7**
- Interactive rebase . . . 7
- Git reset . . . 7

Git stash **8**

Git reflog **8**

Git tags **8**

Git submodules **9**

Git LFS **10**
- Set up Git LFS . . . 10
- Track and untrack files with Git LFS . . . 10
- Download LFS-tracked files to local cache . . . 10
- Clear local cache . . . 10
- Convert an existing repo to Git LFS . . . 10

Configuring Git

Depending on their scope, Git configurations apply to all Git repositories or only to a specific repository. The main 3 scopes are:

- **Global (user wide):** settings apply to all Git repositories controlled by the user. Stored in `~/.gitconfig`, i.e. `/home/<username>/.gitconfig` (Linux), `/Users/<username>/.gitconfig` (macOS) or `C:\Users\<username>\.gitconfig` (Windows). To save a setting as part of the global scope, add the `--global` flag to the `git config` command.
- **Local (repo specific):** settings apply only to a specific Git repo. Stored in the `.git/config` file of the repository.
- **System (system wide):** settings apply to all users and all repos on a given machine. This can only be modified by a system administrator. On Linux, stored in `/etc/gitconfig`. On macOS and Windows, the location depends on how Git was installed (the command below shows it).

To show the list of all your Git configurations, along with their scope and the location of the file they are stored in:

```
git config --list --show-origin
git config --list --show-origin --show-scope  # Only works with Git versions >= 2.26
```

User name and email

Set user name and email. This is the minimum Git config needed to make commits.

```
# Set user name/email globally (for all Git projects of the current user).
git config --global user.name <user name>
git config --global user.email <email>

# Set a user name/email only for the current Git repo.
git config user.name <user name>
git config user.email <email>
```

Retrieve the value set for user name/email (also works for other settings).

```
git config --get user.name
git config --get user.email
```

Change default editor

By default, Git uses the editor set in the `EDITOR` environment variable or, if that variable is not set, `vi` (usually `vim`). This can be changed with the `core.editor` setting.

```
git config --global core.editor nano           # Set "nano" as default editor.
git config --global core.editor vim            # Set "vim" as default editor.
git config --global core.editor "code --wait"  # Set "VS Code" as default editor.

# Retrieve the value set for default editor.
git config --get core.editor
```

Creating Git command aliases

Create a new Git command alias named `<alias name>`, which can then be called with `git <alias name>`.

```
git config --global alias.<alias name> "<command and options to alias>"

# Example: create a "git adog" alias. Note that the alias value does not start with "git".
git config --global alias.adog "log --all --decorate --oneline --graph"
```

Aliases created with `--global` are stored in the `~/.gitconfig` file. One possibility to see the list of existing aliases is using: `git config --list | grep ^alias`

Initializing a new repo and making commits

Create a new Git repository

```
git clone <repo URL>  # Clone an existing remote repository.
git init              # Create a new local repo from scratch.
```

Staging: add and remove content from the index

```
git add <file or directory>   # Stage a file or directory.
git add -u                    # Stage changes to all tracked files (same as --update).
git add --all                 # Stage all changes, including untracked files.

git restore --staged <file>   # Unstage changes to a file.
git restore <file>            # Reset a file to the state it has in the index.
                              # Warning: this discards any uncommitted changes in the file.
git reset HEAD                # Unstage all newly staged content from index.

git rm <file>                 # Delete file from both working tree and index.
git rm --cached <file>        # Delete file from index only (stop tracking it).

git mv <file> <new location>  # Move, rename or move+rename a file.
```

Making new commits

```
git commit                              # Make a new commit (interactive message).
git commit -m "commit message"          # Make a new commit with message "commit message".
git commit -m "commit message" <file>   # Commit only <file> (must already be tracked).
                                        # Other staged changes are not committed.
git commit -am "commit message"         # Shortcut: stage all changes to tracked files and commit.
```

Amend the latest commit of the current branch

```
git commit --amend            # Re-write latest commit with the currently staged content.
git commit --amend --no-edit  # Same as above but keeping the current commit message.
git commit --amend -m "msg"   # Same as above but replacing the commit message with "msg".
```

Display file status, show content differences and history

```
git status                # Show status of files in the working tree.
git show                  # Show last commit on current branch. Same as "git show HEAD".
git show <commit ref>     # Show content of specified commit. E.g. "git show HEAD~1".
git show --name-only      # Show only the names of modified files.
git show --stat           # Same as "--name-only", but additionally show counts of
                          # line changes per file.

git diff                  # Show differences between working tree and Git index.
git diff --cached         # Show differences between Git index and the current HEAD.
git diff --staged         # Same as "git diff --cached".
git diff --name-only      # Show only the names of modified files, not the full diff.
```

```
git diff <ref 1> <ref 2>   # Show differences between commit reference 1 and 2.
                           # E.g. "git diff HEAD~3 HEAD~1".

git log                                      # Display history of current branch.
git log --oneline                            # Condensed history of current branch.
git log --all --decorate --oneline --graph   # Display entire history of repo.

git ls-files     # List files present in the Git index, i.e. the currently tracked files.
                 # Files that were deleted earlier in the history are not listed.

# List all files that are part of the repository's history, i.e. all files in
# any commit of any branch.
git log --all --pretty=format: --name-only --diff-filter=A | sort -u
```

Ignoring files

To ignore files, add their name or pattern to `.gitignore` (shared) or `.git/info/exclude` (private - for files to ignore only in your local repo).

The following are commands to add content to these files directly from the command line (but you can also edit them with a graphical text editor):

```
echo "file name or pattern" >> .gitignore          # Add file name or pattern to the
                                                   # shared .gitignore list.
echo "file name or pattern" >> .git/info/exclude   # Add file name or pattern to the
                                                   # private ignore list.
```

Branches

Create and switch branches

```
git branch                              # List local branches.
git branch -a                           # List both local and remote branches.
git branch <branch name>                # Create a new branch rooted at the current HEAD.
git branch <branch name> <commit ref>   # Create a new branch rooted at a specific commit.

git switch <branch name>     # Switch to a different branch. E.g. "git switch main"
git switch -c <new branch>   # Shortcut: create <new branch> and switch to it.

# Creating a local branch from a remote branch: in this case, a simple switch
# command is sufficient - no need for the -c option.
git switch <branch name>
```

> **Note:** the `git switch` command does not exist in Git versions < 2.23. `git checkout <branch name>` can be used instead (and `git checkout -b <new branch>` to create and switch to a new branch).

Delete branches

Delete local branches:

```
git branch -d <branch name>  # Delete local branch <branch name>.
git branch -D <branch name>  # Force-delete local branch with unmerged changes.
```

Delete remote branches:

```
git push origin --delete <branch>   # Delete the specified branch on the "origin" remote.
```

Merge, rebase and cherry-pick

- To **merge** branch B into branch A, one must be on branch A.
- To **rebase** branch B onto branch A, one must be on branch B.

```
git merge <branch name>            # Merge <branch name> into the current branch.
git merge --abort                  # Abort a merge (e.g. in case of conflicts).

git rebase <branch to rebase on>   # Rebase the current branch onto the latest commit of
                                   # <branch to rebase on>.
git rebase --continue              # Continue a rebase after resolving a conflict.
git rebase --skip                  # Skip the current commit during a rebase.
git rebase --abort                 # Abort a rebase and go back to the state before it.

git cherry-pick <commit ref>       # Cherry-pick commit <commit ref> onto the current
                                   # branch or HEAD.
```

Retrieve content from earlier commits

```
git restore --source=<commit ref> <file>   # Retrieve the specified <file> from commit
                                           # <commit ref>. Warning: this will overwrite the
                                           # current version of <file> in the working tree.
git checkout <commit ref> <file>           # Same as above, but <file> is also overwritten
                                           # in the index.

git checkout <commit ref>    # Set entire working tree content to its state as recorded in
                             # <commit ref>. After this command you will be in "detached
                             # HEAD" state, which you can exit again by switching back to
                             # one of your branches.
```

Working with remotes

Fetch/pull new content from a remote

- The `fetch` command does not modify anything in the working tree, it only retrieves new commits or references (branches, tags) from the remote.
- By default, `fetch`, `pull`, `push` retrieve/push content from/to the **default remote** (generally named `origin`). If you have more than one remote, a non-default remote can be specified.
- If the local branch and its upstream branch on the remote have diverged, recent versions of Git stop with an error when running `git pull`, unless the `pull.rebase` setting is configured. In that case, specify whether to rebase (`git pull --rebase`) or merge (`git pull --no-rebase`).

```
git fetch             # Fetch all new changes from the remote that are not yet in your local repo.
git fetch --prune     # Fetch, then delete the remote-tracking branches (e.g. origin/<branch>)
                      # of branches that no longer exist on the remote.

git pull              # Fetch + update the current branch with the new changes.
git pull --rebase     # Fetch + rebase the current branch onto its upstream branch.
git pull --no-rebase  # Fetch + merge the upstream branch into the current branch.
git pull --prune      # Pull, then delete the remote-tracking branches of branches that
                      # no longer exist on the remote.
```

```
# Fetch/pull content from a specific remote. Useful if there is more than one remote.
git fetch <remote name>
git pull <remote name>
```

Push new content to a remote

```
git push                        # Push new content on current branch to default remote.
git push -u origin <branch>     # Set origin/<branch> as upstream for the current branch, and push
                                # new content on the current branch to the remote. Only needed the
                                # first time a new branch is pushed.
                                # Note: -u is the short option for "--set-upstream".

git push --force-with-lease     # Push the local branch, overwriting the remote branch.
                                # As a safety, raises an error if the remote branch has changed
                                # compared to the last time content from the remote was fetched.
git push --force                # Same as --force-with-lease, but does not check for changes
                                # on the remote since the last sync occurred.

# Push content to a specific remote. Useful if there is more than one remote.
git push <remote name>
```

Add or update a remote

Associate a remote (in this example, from GitHub) with an existing repo. In the examples below, we name the remote `origin` (the default name used by Git), but a remote can be named anything.

- `git remote add`: add an online repository as a remote.
- `git remote set-url`: modify the URL of a remote when a remote is already set.

```
# Add a remote.
git remote add origin https://github.com/<GitHub user name>/<repo name>.git

# Update the URL of an existing remote.
git remote set-url origin https://github.com/<GitHub user name>/<repo name>.git

git remote -v   # Show name and URL of all remotes associated with the repo.
```

Relative references to commits

Here the examples are given with `HEAD`, but an absolute hash, a tag or branch name can also be used.

Reference to an ancestor commit.

```
HEAD~     # Parent of current HEAD.
HEAD~2    # Grand-parent of current HEAD.
HEAD~3    # Great grand-parent of current HEAD.
...
HEAD~X    # Xth generation before current HEAD (general form).
```

Reference to the first or second direct parent.

```
HEAD^1    # First direct parent of HEAD.
HEAD^2    # Second direct parent of HEAD (only exists for merge commits).
```

Detached HEAD state

Detached HEAD state is when the HEAD pointer no longer points to a branch, but instead points directly at a commit.

- Detached HEAD mode is entered after running `git checkout <commit ref>`, where `<commit ref>` is a reference to a commit that is *not* a branch.
- To exit detached HEAD state, simply switch back to a regular branch with `git switch <branch>` (or `git checkout <branch>` if using an older version of Git).
- Adding commits in detached HEAD mode can result in them getting orphaned (i.e. "lost" because they are no longer part of the history) when moving back to a regular branch. To preserve these commits, they must be made part of the history of the repo by either:
  - Creating a new branch rooted at the current position of HEAD (`git switch -c <new branch>` or `git checkout -b <new branch>`).
  - Adding a tag (`git tag <tag name>`).

Editing the Git history

Interactive rebase

Same as a regular rebase but with the possibility to re-order, squash, reword, edit and delete commits.

```
git rebase -i <commit ref>   # Interactively rebase the current branch onto <commit ref>.
git rebase -i --root         # Interactively rebase all commits of the current branch,
                             # including the first one.
```

Fixup commits: create a commit that fixes an earlier commit, then let an interactive rebase squash it into the commit it fixes.

```
git commit --fixup <commit ref>           # Create a commit that fixes <commit ref>.
git rebase -i --autosquash <commit ref>   # Squash fixup commits into the commits they fix.
```

Git reset

Moves the current branch and **HEAD** to the selected reference (e.g. commit, branch, tag) and, depending on the selected option, also resets the Git index and working tree:

- `--mixed`: the default option (passing no option is the same as `--mixed`).
- `--hard`: will make you **lose any uncommitted edits** in your working tree. Use it carefully.

```
git reset --soft  <commit ref>   # --soft: resets HEAD, but neither the index nor the working tree.
git reset --mixed <commit ref>   # --mixed: resets HEAD and index, but not the working tree.
git reset --hard  <commit ref>   # --hard: resets HEAD, index and working tree.
```

Examples (frequent use cases):

```
# Clear staging area (Git index) from newly added material.
git reset HEAD

# Squash the 2 latest commits on the current branch into a single commit.
git reset --soft HEAD~2
git commit -m "the last two commits are now squashed"

# Undo a merge or another reset (i.e. go back to the previous position of HEAD):
git reset --hard HEAD@{1}

# Reset a rebase:
```

```
git reset --hard <ref of latest commit of the rebased branch before the rebase>

# Reset a branch so that it matches the state of its upstream branch. Note that in this
# example the remote is named "origin" (default name for remotes).
git fetch
git reset --hard origin/<branch name>
```

Git stash

`git stash` lets you temporarily clean the working tree by stashing away uncommitted changes (both staged and unstaged), which can then be restored at a later time.

```
git stash         # Stash all uncommitted changes (staged and unstaged) of tracked files.
git stash -u      # Same as above, but also stash untracked files.
git stash list    # List the content of the stash.
git stash pop     # Apply the latest stashed changes to the working tree, and remove them
                  # from the stash.
git stash apply   # Same as "git stash pop", but keep the changes in the stash.
git stash clear   # Delete all stashed content from the stash.
```

Git reflog

The **reflog** is a log of all operations (and the associated `HEAD` positions) performed in the Git repository. The `HEAD@{x}` notation can be used to access the position of `HEAD` relative to the reflog.

The reflog is local: it is not shared when pushing to a remote. Old entries are deleted after some time (by default, after 30 days for commits that are no longer part of a branch, and after 90 days for all other entries).

```
git reflog     # Print the Git reflog.
HEAD@{0}       # Current position of HEAD.
HEAD@{1}       # Position of HEAD one reflog record ago.
HEAD@{2}       # Position of HEAD two reflog records ago.
HEAD@{x}       # Position of HEAD x reflog records ago.
```

Git tags

```
git tag <tag name>               # Create a new lightweight tag (no associated message).
git tag -a <tag name> -m "msg"   # Create a new annotated tag with a tag message.

git tag               # List tags present in the Git repo.
git tag -l "v1.*"     # List only the tags matching a specific pattern, e.g. tags starting
                      # with "v1." in this case.
git tag -n            # Also display tag annotations. For lightweight tags, the commit
                      # message of the commit to which the tag is pointing is shown.

git show <tag name>   # Display changes introduced at the commit referenced by the tag.
                      # For annotated tags, the tagger and tag message are also shown.

git push origin <tag name>   # Push the specified tag to the "origin" remote.
git push --tags              # Push all tags to the default remote.

git tag -d <tag name>                  # Delete tag in local repo.
```

```
git push origin --delete <tag name>    # Delete a tag on the "origin" remote.

git checkout <tag name>   # Check out the commit referenced by the tag.
                          # This will enter "detached HEAD" mode.
```

Note: a commit referenced by a tag will not be garbage-collected, even if it is not part of a branch.

Git submodules

Add/register a new submodule to a project. Submodules are added by using the URL of their online repository (they can also be added without an online remote but it's more complicated).

By default, the submodule is added in a directory named after the repository (e.g. `https://github.com/<user>/tools.git` is added as `tools`), but a different path can also be given.

```
git submodule add <repository URL>          # Add a new submodule to the Git repo.
git submodule add <repository URL> <path>   # Add a new submodule at a custom path.

git mv <submodule path> <new path>          # Move or rename a submodule.
```

Clone a repo with submodules.

```
git clone --recurse-submodules <repo URL>   # Clone a repository containing submodules.

# The above is a shortcut for:
git clone <repo URL>
git submodule update --init --recursive   # Initialize all submodules (including nested
                                          # submodules) and download their content.
```

Work with submodules.

```
git submodule status                     # Show the commit checked out in each submodule.
git diff --submodule                     # Show the list of new commits in submodules.
git diff --submodule=diff                # Show the actual changes in submodule files.
git pull --recurse-submodules            # Pull the main repo, then update its submodules
                                         # to the commits recorded in the main repo.

git submodule foreach "<command>"        # Execute the specified command in all submodules.
git submodule update --init --recursive  # Initialize submodules that are not yet initialized,
                                         # then check out, in each submodule, the commit
                                         # recorded in the main repo. This leaves the
                                         # submodules in "detached HEAD" state.

git push --recurse-submodules=check      # Only make a push on the main repo if all changes
                                         # in the submodules have been pushed.
git push --recurse-submodules=on-demand  # Try to push the main project together with all
                                         # submodules.
git submodule update --remote --merge    # Get the latest versions of your submodules and
                                         # merge them with your changes. By default, the
                                         # default branch of the submodule's remote is used.
                                         # Another branch can be specified in .gitmodules.
```

Git LFS

Set up Git LFS

```
git lfs install   # Set up Git LFS for the current user. Only needs to be run once.
```

Track and untrack files with Git LFS

```
git lfs track <file name>       # Add file name/pattern to list of LFS-tracked files,
                                # i.e. add them to the .gitattributes file.
git lfs track "*.pattern"       # To track a pattern, it must be quoted, so that the
                                # shell does not expand it.
git lfs track <directory/**>    # To track the content of an entire directory (recursively),
                                # the directory name must be followed by "/**".

git lfs untrack <file name or pattern>   # Stop LFS-tracking on file name/pattern. The
                                         # same can be achieved by deleting a line in
                                         # the .gitattributes file.

git lfs ls-files               # List LFS-tracked files associated with the HEAD commit.
git lfs ls-files <commit ref>  # List LFS-tracked files associated with a specific
                               # commit hash or reference.
git lfs ls-files --all         # List the LFS-tracked files of all commits (the entire history).
```

Download LFS-tracked files to local cache

```
git lfs fetch --recent  # Download all LFS-tracked files associated with "recent" commits.
git lfs fetch --all     # Download all LFS-tracked files to the local cache.
```

Clear local cache

To save space on the local machine, LFS-tracked files that are locally cached can be deleted. They will remain available on the remote LFS object store (provided they were pushed at some point).

```
git lfs prune                    # Delete files that are no longer needed from the local LFS cache.
git lfs prune --dry-run          # Same as above, but in "test" mode: only show what
                                 # would be deleted, without actually deleting it.
git lfs prune --verify-remote    # Double-check that LFS-tracked files are really present
                                 # on the remote object store before deleting them.
```

Convert an existing repo to Git LFS

> **Warning:** this will rewrite the entire history of the Git repository! It's strongly advised to make a copy of the repo before running this command.

```
# Several patterns can be given, separated by commas, e.g. --include="*.tif,*.jpg"
git lfs migrate import --include="<file name or pattern>" --everything

git lfs checkout   # After a "migrate import" command, the LFS-tracked files in the
                   # working tree have their content replaced by the pointer content.
                   # The "lfs checkout" command restores their original content.

# After the migration, the old copies of the files remain in the Git object store until
# Git's automatic garbage collection deletes them. To delete them immediately:
git reflog expire --expire-unreachable=now --all
git gc --prune=now
```